Batch: B2	Roll No.: 16010124107
Experiment No. 05	
Grade: AA / AB / BB / BC / CC / CD /DD	
Signature of the Staff In-charge with date	

TITLE: Implementation of Basic Process management algorithms - Preemptive (RR)

AIM: To implement basic Process management algorithm (Round Robin)

Expected Outcome of Experiment:
CO 2. To understand the concept of process, thread and resource management.

Books/ Journals/ Websites referred:

1. **Silberschatz A., Galvin P., Gagne G. "Operating Systems Principles", Willey Eight edition.**
2. **Achyut S. Godbole , Atul Kahate "Operating Systems" McGraw Hill Third**
3. **Edition.**
4. **William Stallings, "Operating System Internal & Design Principles", Pearson.**
5. **Andrew S. Tanenbaum, "Modern Operating System", Prentice Hall.**

Pre Lab/ Prior Concepts:

Most systems handle numerous processes with short CPU bursts interspersed with I/O requests and a few processes with long CPU bursts. To ensure good time-sharing performance, a running process may be preempted to allow another to run. The ready list, or run queue, maintains all processes ready to run and not blocked by I/O or other system requests. Entries in this list point to the process control block, which stores all process information and state.

When an I/O request completes, the process moves from the waiting state to the ready state and is placed on the run queue. The process scheduler, a key component of the operating system, decides whether the current process should continue running or if another should take over. This decision is triggered by four events:

1. The current process issues an I/O request or system request, moving it from running to waiting.
2. The current process terminates.
3. A timer interrupt indicates the process has run for its allotted time, moving it from running to ready.

4. An I/O operation completes, moving the process from waiting to ready, potentially preempting the current process.

The scheduling algorithm, or policy, determines the sequence and duration of process execution, a complex task given the limited information about ready processes.

Description of the application to be implemented:

Round Robin Algorithm

Round Robin (RR) is a preemptive CPU scheduling algorithm designed for time-sharing systems. Each process is assigned a fixed time quantum and is executed for that duration. If a process doesn't finish within its time quantum, it is preempted and placed at the end of the ready queue, allowing the next process to run. RR ensures fairness, prevents process starvation, and gives each process regular access to the CPU, but its performance depends on the choice of the time quantum: too small increases context switching, too large behaves like FCFS.

Code:

```
#include <iostream>

#include <queue>

using namespace std;


struct Process {

   int pid, at, bt, rt, ct, wt, tat;

};


int main() {

   int n, tq;

   cout << "Enter number of processes: ";

   cin >> n;


   Process p[n];

   for(int i = 0; i < n; i++) {

      p[i].pid = i + 1;
```

```
    cout << "Enter Arrival Time for P" << i+1 << ": ";
    cin >> p[i].at;
    cout << "Enter Burst Time for P" << i+1 << ": ";
    cin >> p[i].bt;
    p[i].rt = p[i].bt;
}

cout << "Enter Time Quantum: ";
cin >> tq;

queue<int> q;
int currentTime = 0, completed = 0;
bool visited[n] = {0};

for(int i = 0; i < n; i++) {
    if(p[i].at <= currentTime) {
        q.push(i);
        visited[i] = true;
    }
}

while(completed < n) {
    if(q.empty()) {
        currentTime++;
        for(int i = 0; i < n; i++) {
            if(!visited[i] && p[i].at <= currentTime) {
                q.push(i);
                visited[i] = true;
```

```
            }
        }
        continue;
    }

    int idx = q.front();
    q.pop();

    int execTime = (p[idx].rt >= tq) ? tq : p[idx].rt;
    p[idx].rt -= execTime;
    currentTime += execTime;

    for(int i = 0; i < n; i++) {
        if(!visited[i] && p[i].at <= currentTime) {
            q.push(i);
            visited[i] = true;
        }
    }

    if(p[idx].rt > 0) {
        q.push(idx);
    } else {
        p[idx].ct = currentTime;
        p[idx].tat = p[idx].ct - p[idx].at;
        p[idx].wt = p[idx].tat - p[idx].bt;
        completed++;
    }
}
```

```
    float total_wt = 0, total_tat = 0;
    cout << "\nP\tAT\tBT\tCT\tWT\tTAT\n";
    for(int i = 0; i < n; i++) {
        cout << "P" << p[i].pid << "\t" << p[i].at << "\t" << p[i].bt << "\t"
            << p[i].ct << "\t" << p[i].wt << "\t" << p[i].tat << endl;
        total_wt += p[i].wt;
        total_tat += p[i].tat;
    }

    cout << "\nAverage Waiting Time: " << total_wt/n;
    cout << "\nAverage Turnaround Time: " << total_tat/n;

    return 0;
}
```

Output:

```
Enter number of processes: 5
Enter Arrival Time for P1: 0
Enter Burst Time for P1: 5
Enter Arrival Time for P2: 2
Enter Burst Time for P2: 10
Enter Arrival Time for P3: 4
Enter Burst Time for P3: 5
Enter Arrival Time for P4: 6
Enter Burst Time for P4: 3
Enter Arrival Time for P5: 8
Enter Burst Time for P5: 2
Enter Time Quantum: 2

P       AT      BT      CT      WT      TAT
P1      0       5       13      8       13
P2      2       10      25      13      23
P3      4       5       21      12      17
P4      6       3       20      11      14
P5      8       2       15      5       7

Average Waiting Time: 9.8
Average Turnaround Time: 14.8
```

Conclusion:

In conclusion, the Round Robin scheduling algorithm ensures fairness by giving each process a fixed time slice and cycling through the ready queue. It effectively prevents starvation and is suitable for time-sharing systems, but its performance depends on the chosen time quantum: too small increases context switching overhead, while too large reduces responsiveness.

Post Lab Descriptive Questions

1. What effect the time quantum has on its performance. What are the advantages and disadvantages of using a small versus a large time quantum?
   The time quantum in Round Robin affects performance significantly. A small quantum increases responsiveness, letting short processes finish quickly, but causes high context switching, lowering CPU efficiency. A large quantum

reduces switching overhead and improves CPU utilization, but short processes wait longer and system responsiveness drops, behaving like FCFS. Optimal quantum balances responsiveness and efficiency, avoiding excessive overhead while keeping processes fairly scheduled.

2. The following processes are scheduled using the Robin process scheduling policy with a time quantum of 3ms. Determine the average waiting time.

Process	Arrival Time	Burst Time	Remaining Time
P1	0	6	6
P2	1	2	2
P3	3	8	8
P4	5	3	3
P5	2	4	4

Time Quantum (TQ) = 3 ms

We execute processes in ready queue order, preempting after 3 ms or if the process finishes earlier.

1. t = 0 → 3: P1 runs 3 ms (RT = 3)
   Queue: P2 arrived at t = 1 → added, P5 arrived at t = 2 → added
2. t = 3 → 5: P2 runs 2 ms (finished)
   Queue: P5, P3 (arrived t = 3)
3. t = 5 → 8: P5 runs 3 ms (RT = 1)
   Queue: P3, P4 (arrived t = 5), P1 (RT = 3), P5 (RT = 1)
4. t = 8 → 11: P3 runs 3 ms (RT = 5)
   Queue: P4, P1, P5, P3
5. t = 11 → 14: P4 runs 3 ms (finished)
   Queue: P1, P5, P3
6. t = 14 → 17: P1 runs 3 ms (finished)
   Queue: P5, P3
7. t = 17 → 18: P5 runs 1 ms (finished)
   Queue: P3
8. t = 18 → 21: P3 runs 3 ms (RT = 2)
9. t = 21 → 23: P3 runs 2 ms (finished)

Process	CT	TAT=CT-AT	WT
P1	17	17	11

P2	5	4	2
P3	23	20	12
P4	14	9	6
P5	18	16	12

Average waiting time= 43/5 = 8.6

Date: 01/04/2026 **Signature of faculty in-charge**